Variables and Data Types in C

1. What is a Variable?

A **variable** is a named memory location used to store data during program execution.

For example:

int age = 25;

Here:

- int → data type
- age → variable name
- 25 → value
- Memory is allocated to store the value.

Conceptually:

Memory

```
+----------------+
|     25      |
+----------------+
     ↑
    age
```

The data type tells the compiler **what kind of data the variable stores and how it should be interpreted**.

---

2. Basic Data Types in C

The fundamental C data types are:

Data Type	Typical Size	Typical Range / Precision
char	1 byte	-128 to 127 or 0 to 255
short	2 bytes	-32,768 to 32,767
int	4 bytes	-2,147,483,648 to 2,147,483,647
long	4 or 8 bytes	Implementation dependent

Data Type	Typical Size	Typical Range / Precision
long long	8 bytes	approximately ±9.22 × 10^{18}
float	4 bytes	~6–7 decimal digits
double	8 bytes	~15–16 decimal digits
long double	8, 12, or 16 bytes	Implementation dependent
_Bool	typically 1 byte	0 or 1

Important: C does not guarantee that int is always 4 bytes or long is always 8 bytes. The exact sizes depend on the compiler and platform.

---

3. Integer Data Types

Integer types store whole numbers.

char

char letter = 'A';

Usually:

1 byte = 8 bits

A char can represent a character because characters are internally represented by integer codes.

For example, ASCII:

'A' = 65

'B' = 66

'a' = 97

Example:

char c = 'A';

printf("%c\n", c);

printf("%d\n", c);

Output:

A

---

4. short

short age = 25;

Usually:

2 bytes = 16 bits

For a signed 16-bit integer:

−215≤x≤215−1-2^{15} \leq x \leq 2^{15}-1

Therefore:

-32,768 ... 32,767

---

5. int

The most commonly used integer type is int.

int number = 100;

On most modern systems:

int = 4 bytes = 32 bits

Signed range:

−231 to 231−1-2^{31} \text{ to } 2^{31}-1

Therefore:

-2,147,483,648

to

2,147,483,647

Example:

int population = 1000000;

---

6. long

long distance = 1000000000L;

The size of long is **platform dependent**.

Common situations:

Windows 64-bit

long = 4 bytes

Linux/macOS 64-bit

long = 8 bytes

This is an important issue when teaching C.

Do not tell students:

long is always 8 bytes.

That is incorrect.

---

7. long long

long long population = 9000000000LL;

Usually:

8 bytes = 64 bits

Range:

−263 to 263−1-2^{63} \text{ to } 2^{63}-1

Approximately:

-9.22 × 10^18

to

+9.22 × 10^18

---

8. Signed and Unsigned Types

C allows integer types to be declared as:

- signed
- unsigned

For example:

int x;

signed int y;

unsigned int z;

int is signed by default.

Signed

A signed 32-bit integer:

-2,147,483,648

...

2,147,483,647

Unsigned

unsigned int x;

Typically:

0

...

4,294,967,295

Why?

A 32-bit unsigned integer has:

2322^{32}

possible values:

0...232−10 \ldots 2^{32}-1

---

9. Important Integer Table

Type	Typical Size	Signed Range
signed char	1 byte	−128 to 127
unsigned char	1 byte	0 to 255
short	2 bytes	−32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
int	4 bytes	-2^{31} to $2^{31}-1$
unsigned int	4 bytes	0 to $2^{32}-1$

Type	Typical Size	Signed Range
long long	8 bytes	-2^{63} to $2^{63}-1$
unsigned long long	8 bytes	0 to $2^{64}-1$

---

10. Floating-Point Data Types

C provides three main floating-point types:

float

double

long double

They are used to represent numbers with fractional parts.

For example:

float temperature = 25.5;

double pi = 3.141592653589793;

---

float

Typically:

4 bytes = 32 bits

Approximately 6–7 significant decimal digits.

float x = 3.1415926f;

The f suffix tells the compiler that the constant is a float.

---

double

Typically:

8 bytes = 64 bits

Approximately 15–16 significant decimal digits.

double pi = 3.141592653589793;

double is generally preferred when greater precision is required.

---

long double

The size is implementation dependent.

It may be:

8 bytes

12 bytes

16 bytes

depending on compiler and architecture.

Example:

long double value = 3.141592653589793238L;

---

11. _Bool

C has a Boolean type:

_Bool

Example:

_Bool isStudent = 1;

Values are:

0 → false

1 → true

With:

#include <stdbool.h>

we can use the more convenient:

bool

true

false

Example:

#include <stdio.h>

#include <stdbool.h>

```
int main()
{
   bool passed = true;

   printf("%d\n", passed);

   return 0;
}
```

Output:

1

---

12. How to Determine the Actual Memory Size

The sizeof operator is used to determine the size of a data type or variable.

sizeof(int)

Example:

```
#include <stdio.h>

int main()
{
   printf("char      : %zu bytes\n", sizeof(char));
   printf("short     : %zu bytes\n", sizeof(short));
   printf("int       : %zu bytes\n", sizeof(int));
   printf("long      : %zu bytes\n", sizeof(long));
   printf("long long  : %zu bytes\n", sizeof(long long));

   printf("float     : %zu bytes\n", sizeof(float));
   printf("double    : %zu bytes\n", sizeof(double));
   printf("long double: %zu bytes\n", sizeof(long double));
```

```
    return 0;
}
```

On a typical 64-bit Windows compiler, you might see:

```
char      : 1 bytes
short     : 2 bytes
int       : 4 bytes
long      : 4 bytes
long long  : 8 bytes

float     : 4 bytes
double    : 8 bytes
long double: 8 bytes
```

But the output can differ on another platform.

---

13. Why Does sizeof(char) Always Equal 1?

This is a particularly important concept.

In C:

sizeof(char)

is **always 1**.

However, C defines the size of a char as one **byte**, and a C byte is not theoretically required to be exactly 8 bits.

The number of bits in a byte is given by:

CHAR_BIT

from:

#include <limits.h>

Example:

#include <stdio.h>

```
#include <limits.h>


int main()
{
   printf("Bits in a byte = %d\n", CHAR_BIT);
}
```

On virtually all modern desktop systems:

```
Bits in a byte = 8
```

---

14. Variables and Memory

Consider:

```
int age = 25;
double salary = 50000.75;
char grade = 'A';
```

Conceptually:

```
Variable     Type      Typical memory


age          int       4 bytes
salary       double    8 bytes
grade        char      1 byte
```

The compiler allocates memory for these variables.

For example:

```
Memory
+------------------+
|                  |
| age = 25         | 4 bytes
|                  |
+------------------+
```

| |

| salary | 8 bytes

| 50000.75 |

| |

+------------------+

| |

| grade = 'A' | 1 byte

| |

+------------------+

The actual addresses are determined by the compiler/runtime environment.

---

15. Variable Declaration

A variable can be declared as:

int age;

This tells the compiler:

Create an integer variable called age.

We can then assign a value:

age = 25;

Or combine declaration and initialization:

int age = 25;

---

16. Multiple Variables

C allows:

int x, y, z;

and:

int x = 10, y = 20, z = 30;

However, for readability, especially in teaching and professional programming, this is usually clearer:

```
int x = 10;

int y = 20;

int z = 30;
```

---

17. Constants vs Variables

A variable can change:

```
int age = 20;


age = 21;
```

A constant cannot normally be modified:

```
const int MAX = 100;
```

This is illegal:

```
MAX = 200;   // error
```

---

18. Format Specifiers

Each data type normally has an appropriate printf format specifier.

Type	printf
char	%c
int	%d
unsigned int	%u
short	%hd
long	%ld
long long	%lld
float	%f
double	%f
long double	%Lf

Example:

```
int age = 25;

float height = 1.75f;

double salary = 50000.50;

char grade = 'A';


printf("Age    = %d\n", age);

printf("Height = %f\n", height);

printf("Salary = %f\n", salary);

printf("Grade  = %c\n", grade);
```

---

19. Memory Size vs Range

A key concept for students is:

The larger the number of bits available, the more different values can generally be represented.

For an unsigned integer with n bits:

Number of possible values=2n\text{Number of possible values}=2^n

For example:

8 bits

28=2562^8=256

Values:

0 – 255

16 bits

216=65,5362^{16}=65,536

Values:

0 – 65,535

32 bits

232=4,294,967,2962^{32}=4,294,967,296

Values:

0 – 4,294,967,295

64 bits

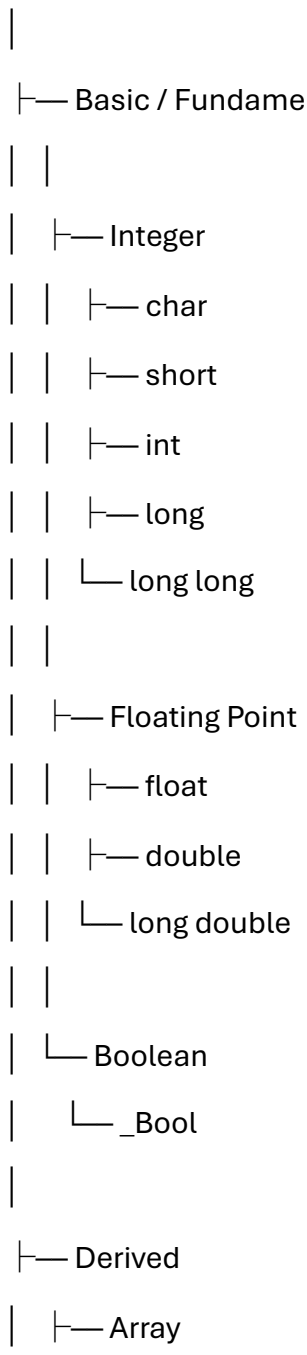
264=18,446,744,073,709,551,6162^{64}=18,446,744,073,709,551,616

---

20. Important C Data-Type Hierarchy

A useful way to present the subject to students is:

C Data Types



```
│   ├── Pointer
│   └── Function
│
├── User-defined
│   ├── struct
│   ├── union
│   └── enum
│
└── void
```

---

21. Recommended Lecture Table

For a C programming course, I would give students this summary:

Data Type	Typical Size	Example	Typical Use
char	1 B	'A'	Character
short	2 B	1000	Small integers
int	4 B	100000	General integers
long	4/8 B	1000000L	Larger integers
long long	8 B	9000000000LL	Very large integers
float	4 B	3.14f	Decimal, lower precision
double	8 B	3.14159265	Decimal, higher precision
long double	implementation-dependent	3.14L	Extended precision
_Bool	typically 1 B	1	True/False
void	no storage	—	No value/type

One important teaching point

Do not teach memory sizes as absolute facts in C. Teach students to distinguish between:

1. **minimum guarantees defined by the C standard**, and

2. **typical sizes on their particular compiler/platform**.

The correct way to ask the computer is:

printf("%zu\n", sizeof(int));

rather than assuming int is always 4 bytes.